Exam

Pointers, dynamic arrays and linked lists

Duration: 30 minutes · 20 points · No documents, no computer

Name: _____ Student number: _____

Instructions. Write your answers on this sheet. Code must be real C: declare your variables and keep the semicolons, but a missing `#include` costs nothing. Suggested timing: Exercise 1 about 8 minutes, Exercise 2 about 10 minutes, Exercise 3 about 12 minutes.

Conventions for every fragment of code in this exam:

- `#include <stdio.h>`, `<stdlib.h>` and `<string.h>` are at the top of every program.
- 64-bit machine: `sizeof(int)` is 4 bytes, `sizeof(void *)` is 8 bytes.
- `malloc` and `realloc` never fail unless the question says otherwise.
- When a question asks what a program prints, write exactly the characters that appear on the screen.

Question:	1	2	3	Total
Points:	6	7	7	20
Score:				

Exercise 1. Pointers (6 points)

1.1 **Trace.** This program compiles and runs without error. Write the four lines it prints, labelled A to D. Then explain in one sentence why the first number of line A is not 10. [3 pt]

```c
#include <stdio.h>

void bump(int *p) { (*p)++; }

int main(void) {
    int a = 10, b = 20;
    int *p = &a;
    int arr[4] = {1, 2, 3, 4};
    int *q = arr;

    *p = *p + 5;
    p = &b;
    *p = 0;
    bump(&a);
    q++;
    *q = 9;
    printf("%d %d\n", a, b);            /* line A */
    printf("%d %d\n", arr[1], *q);      /* line B */
    printf("%d\n", *(arr + 3));         /* line C */
    printf("%d\n", (int)(q - arr));     /* line D */
    return 0;
}
```

1.2 **Three bugs** (1 point each). Each fragment compiles, yet each contains one pointer mistake. For each, say in one sentence what is wrong and what can happen at run time, then give a corrected version (one or two lines). [3 pt]

(a)

```
int *make(void) {
    int x = 42;
    return &x;
}
```

(b)

```
int *p;
*p = 5;
printf("%d\n", *p);
```

(c)

```
char *s = "hello";
s[0] = 'H';
printf("%s\n", s);
```

Exercise 2. Dynamic arrays (7 points)

A growable array of `int` is described by this structure. `size` elements are in use; the block pointed to by `data` has room for `capacity` of them.

```
typedef struct {
    int *data;        /* the elements, on the heap */
    size_t size;      /* how many are in use */
    size_t capacity;  /* how many fit in data */
} IntVec;

IntVec vec_new(void) {
    IntVec v = { NULL, 0, 0 };
    return v;
}
```

2.1 **vec_push.** Write `int vec_push(IntVec *v, int x)`. It appends `x` after the last element in use. When the array is full (`size == capacity`), it first grows the block with `realloc`: a capacity of 0 becomes 4, any other capacity doubles. It returns 0 on success and -1 if `realloc` fails, in which case the vector must be left exactly as it was. [3 pt]

2.2 **By address.** Why does `vec_push` take an `IntVec *` rather than an `IntVec`? What would happen with `int vec_push(IntVec v, int x)`? [1 pt]

2.3 **vec_free.** Write `void vec_free(IntVec *v)`, which releases the memory and leaves the vector empty and safe to push into again. [1 pt]

2.4 **Counting.** After the following line, give `v.size`, `v.capacity` and the number of times `realloc` was called by your `vec_push`. [2 pt]

```
IntVec v = vec_new();  for (int i = 0; i < 10; i++) vec_push(&v, i * i);
```

A student then writes `int n = sizeof(v.data) / sizeof(v.data[0]);` to count the elements. What value does `n` get, and why is this not the number of elements?

Exercise 3. Linked lists (7 points)

A singly linked list of `int` uses this node and this function, which you may call without rewriting it.

```c
typedef struct node {
    int value;
    struct node *next;
} Node;

/* Adds a node at the front; returns the new head. */
Node *push_front(Node *head, int value) {
    Node *n = malloc(sizeof(Node));
    if (n == NULL) return head;
    n->value = value;
    n->next = head;
    return n;
}
```

3.1 **Draw it.** Draw the list after these lines: one box per node with its value, an arrow per `next`, the `head` variable, and `NULL` at the end. [2 pt]

```c
Node *head = NULL;
head = push_front(head, 3);
head = push_front(head, 1);
head = push_front(head, 4);
head->next->next->value = 5;
```

3.2 **length.** Write `size_t length(const Node *head)`, which returns the number of nodes, without recursion. [1 pt]

3.3 **remove_first.** Write `Node *remove_first(Node *head, int value)`. It removes the first node whose `value` equals the argument, frees that node, and returns the head of the resulting list. If no node matches, the list is unchanged. It must work when the matching node is the head, when it is the last node, and when the list is empty. [3 pt]

3.4 **free_list.** A student writes this to free a whole list. Explain what is wrong and fix it. [1 pt]

```
void free_list(Node *head) {
    while (head != NULL) {
        free(head);
        head = head->next;
    }
}
```